Taller parte práctica draw.io
nombre: Sergio David Martinez Ruiz
código: 20237578737
materia: análisis de sistemas

parte ①

Bug a corregir

Error 1: La "Base de datos de Personajes" no es una entidad externa. Es un almacén interno del sistema, así que queda dentro del círculo (rpg manager). Las entidades externas son personas u otros sistemas que viven fuera de él.

Error 2: "Crear Personaje" es un caso (una acción del sistema), no un flujo de datos. Las flechas de un diagrama de contexto deben rotularse con datos que viajan, por ejemplo "datos del personaje" o "nombre y clase elegida".

Error 3: Dos entidades externas no se comunican directamente en un diagrama de contexto. Todo flujo debe pasar por el sistema. Si el jugador y el game master interactúan, lo hacen a través del rpg manager.

Ejercicio 2A generalización de

¿Hacia donde apunta el triangulo vacío y por que?

R= Apunta hacia el sector más general, es decir, hacia jugador. Lo hace porque la generalización se lee como "es un": "el jugador premium es un jugador". El triangulo señala al padre y el hijo hereda lo que el padre puede hacer.

¿Por que el jugador premium no necesita lineas a CU-01 ... CU-07?

R= por que hereda todas las asociaciones del jugador, al ser un tipo de jugador, ya puede registrarse, iniciar sesión, crear personaje, etc. Dibujar esas lineas ensuciaria el diagrama. Solo se dibuja su linea hasta CU-13

Bug a corregir 2.

- Error 1: "Inventario" es un sustantivo, no una acción. Un caso de uso debe nombrarse con verbo + complemento. Por ejemplo "Gestionar inventario".

Powered by CamScanner

-Error 2: El Game master es un actor y los actores van fuera del rectangulo de sistema. Dentro solo van los casos de uso.

-Error 3: El <<include>> está invertido debe ir de CU-03 Crear personaje Personaje(base) hacia CU-04 elegir clase porque con. La flecha apunta al caso incluido.

-Error 4: Los actores no se asocian a menudo entre sí en un diagrama de casos de uso. Los asocian solo un ~~[illegible]~~ actor a caso de uso. La relación permitida entre actores no una asociación directa.

Ejercicio 3 A ¿Y si se acaban las pociones? ¿En que carril va la desición?

R= En el carril RPG manager porque es el sistema quien consulte el inventario y verifica si hay pociones disponibles.

- Si hay → restaurar vida
- Si no hay → informar que no hay magia

③ Bug a corregir (diagrama de actividades)

- Error 1: un rombo de desicion debe tener al menos dos salidas con guardas que cubran todos los casos, falta la salida del caso no.

- Error 2: un fork que abre caminos paralelos debe cerrarse con un join que los sincronice. Si cada camino termina en su propio nodo final, el flujo queda sin sincronizar. Deben converger en un join y de ahí a un único nodo final.

- Error 3: "calcula daño al oponente" es un cálculo que realiza el sistema, así que debe estar en el carril Reg manager no en el de jugador. El jugador solo decide atacar.

④ preguntas de reflexión.

- ¿Por qué el diagrama de contexto no muestra la base de datos si el sistema sí la usa?

Por que el diagrama de contexto muestra el sistema como una caja negra: solo

[illegible]

interese lo que entra y sale entre el sistema y el exterior. La base de datos es un detalle interno de implementación y se menciona en niveles más detallados.

- ¿diferencia entre el flujo "datos de personaje" y el caso de uso "crear personaje"? ⑥

El flujo de datos es la información que viaja entre el actor y el sistema, mientras que el caso de uso es la funcionalidad que el sistema hace. "datos del personaje" es el insumo; "crear personaje" es la acción que lo usa.

⑦ ejemplos propios de include y extend

– <<include>>: comprar un ítem exclusivo siempre se debe validar el saldo para poder comprar, un ítem o misión incluye validar nivel máximo.

– <<extend>>: combatir en la arena es extendido por rendirse. Solo ocurre en ciertos casos y es como una función, sin el a también sirve gestionar inventario extendido por vender ítem

4. ¿Qué aporta el diagrama de actividades? Muestra el orden y la lógica del flujo: la secuencia de pasos, las decisiones y condiciones, los ciclos, el paralelismo y quién hace cada paso gracias a los carriles. El de los casos de uso solo dice que funcionalidades existen y quién las usa, no como se ejecutan.

5. ¿En qué clase conviene calcular daño al oponente?

Clase combate, porque el daño depende de datos de ambos participantes (ataque y defensa) y de sus reglas, clase de combates etc. Otra opción sería en la clase de recibidor dato. como atacar() y el daño recibido calculado en una función llamada recibirdaño(). Y justificar que el método vive donde la información que necesita.

⑥ Cuando draw.io y cuando PlantUML?

- PlantUML: cuando el diagrama se escribe como texto y se guarda en el repositorio junto al código. es ideal para versionar con git, mantener diagramas actualizados fácilmente, generarlos automáticamente y trabajar en equipos técnicos.

- draw.io: cuando necesitas control visual total, diagramas más personalizables o presentaciones para público no técnico y quieres arrastrar y soltar sin aprender sintaxis.